

Spring Framework: Spring Boot

Learning Java Development is About: Learning the following tech

A. Java Language (Core Java)

B. Java Technology (JDBC, Servlet, JSP, JSTL, etc.)

- **Java Database Connectivity (JDBC):** A set of APIs that allow Java programs to interact with databases.
- **Servlets:** Java programs that run on a web server to handle HTTP requests and responses.
- **JSP (Java Server Pages):** Used to create dynamic web content by embedding Java code in HTML.

C. Framework (Hibernate, Spring, Spring Boot, Microservices, REST APIs, etc.)

- **A framework is not a new technology but an abstraction built on top of existing technologies to simplify development.**
- **A framework provides a set of pre-built functionalities that help you build applications more easily.**
- **It reduces the need to write repetitive boilerplate code by offering reusable components and automated tasks, such as object creation, configuration, or database management.**

Spring:

- **Spring is a comprehensive framework for building enterprise-level applications.**
- **Spring manages objects for you. Based on the configuration provided, it creates objects, injects dependencies, and performs object lifecycle management.**

Important Key Features of Spring:

1. Easier Java Development:

- Spring provides a comprehensive set of tools and best practices that make it easier to develop Java applications.
- It simplifies the development process by managing infrastructure concerns (like transactions, security, and database access), allowing developers to focus more on business logic.

2. End-to-End Backend Development:

- Spring can be used to develop the complete backend of an application, from handling user requests in the web layer (Spring MVC) to managing data access (Spring Data), and handling business logic in service layers.
- It is flexible enough to build monolithic applications or microservices-based architectures.

3. Versatility:

- Spring integrates easily with other technologies and frameworks, making it a versatile choice for modern application development.
- For example, it integrates with **Hibernate** and **JPA** for data access, enabling seamless database operations while abstracting complexity.

4. Object Management:

- Spring manages how **objects are created and used**, reducing the complexity of your code by handling object lifecycle and dependency management.
- With **Dependency Injection (DI)**, Spring helps in decoupling components, making the codebase more modular and maintainable.

5. Dependency Injection (DI):

- Spring allows you to **inject dependencies** (pass objects into other objects), which makes your code **flexible, loosely coupled, and easier to test**.
- DI promotes better separation of concerns and easier unit testing of individual components.

6. Modular Structure:

- Spring is organized into a wide array of **modules** (e.g., **Spring MVC** for web applications, **Spring Data** for data access, **Spring Security** for authentication and authorization, etc.).

Spring makes your code more testable, maintainable, and loosely coupled.

Whether you're building a monolithic application or microservices, Spring provides the tools and flexibility to support a wide range of applications with ease.

Spring Core:

- **The Spring Core module is the foundation of all other modules and provides fundamental features such as Dependency Injection, Inversion of Control (IoC), and Bean management.**
- **Dependency Management:** The Spring Core module supplies the Spring container to perform Dependency Management through Inversion of Control (IoC). This helps in decoupling components

Uploaded by @hmaracollege

Dependency Management in Spring

Dependency Management in Spring is the process of assigning **dependent objects** to a **target object** by loading both classes and creating their instances. This is typically done using **Dependency Injection (DI)**, where one class (the dependent class) is injected into another (the target class) to manage the relationships between objects.

1. **Target Class:**
 - The class that depends on the services or functionality provided by another class is called the **target class**.
2. **Dependent Class:**
 - The class that provides the necessary services or functionality to the target class is called the **dependent class**.

Common Modes of Dependency Injection

Setter Injection

- Setter Injection is the process of injecting dependencies into a target class through setter methods.
- The Spring container first creates an instance of the target class, and then calls the appropriate setter method to inject the dependent object.

Constructor Injection

- Constructor Injection involves passing the dependent object to the target object via the constructor.
- This ensures that all required dependencies are provided at the time of object creation, making the object immutable and fully initialized.

Field Injection

- Field Injection involves directly injecting dependencies into the fields of the target class using reflection.
- The Spring container automatically assigns the dependency to the field at runtime.

Spring Containers (IoC Containers)

Uploaded by @hmaracollege

The Spring Core module provides two types of IoC containers:

1. **BeanFactory Container**
2. **ApplicationContext Container** (Latest and most widely used)

Summary of Differences:

Feature	BeanFactory	ApplicationContext
Initialization Type	Lazy Initialization (beans are created when needed)	Eager Initialization (beans are created at startup)
Advanced Features	Basic container, lacks advanced features	Provides additional features like event propagation, and annotation support

Performance	Faster startup with memory efficiency	Slower startup, but better runtime performance and availability of all beans
Example	<code>XmlBeanFactory</code> (deprecated in favor of <code>ApplicationContext</code>)	<code>AnnotationConfigApplicationContext</code> , <code>ClassPathXmlApplicationContext</code>

For most Spring applications today, `ApplicationContext` is recommended due to its richer feature set and better alignment with modern development practices.

Spring Application Development Approaches

Spring applications can be developed using the following approaches:

- 1. XML Configuration (Legacy Approach):**
 - Traditional configuration approach where Spring beans and dependencies are configured in an XML file.
 - Mainly used in legacy or maintenance projects.
- 2. Annotation-Driven Configuration:**
 - A mix of XML and Java configuration using annotations like `@Component`, `@Autowired`, `@Bean`, etc.
 - Easier to manage and more flexible than XML alone.
- 3. Java-Based Configuration:**
 - Configuration is entirely done using Java code (using `@Configuration`, `@Bean` annotations) and does not require XML configuration files.
 - The most modern and widely used approach in Spring development.
- 4. Spring Boot Auto-Configuration:**
 - Supported by Spring Boot, this approach simplifies configuration by automatically configuring Spring beans based on the application's requirements.
 - Spring Boot automatically detects the necessary dependencies and applies the correct configuration, making the process seamless.

Spring Boot:

Spring Boot is one of the most efficient and simplified approaches to working with the Spring Framework, offering a streamlined way to build Spring-based applications with minimal configuration.

Spring Boot = Spring + Auto Configuration + Embedded Servers + Actuators

Spring Boot is a combination of:

- **Spring** (Core framework)
- **XML configurations** (reduced or eliminated)
- **Auto Configuration** (automatic setup based on context)
- **Embedded Servers** (no need to configure an external server)
- **Actuators** (for monitoring and management)

Key Features of Spring Boot:

1. Simplified Application Development:

Spring Boot simplifies the development of Spring-based applications by removing the need for complex configurations. Developers can focus more on business logic and less on infrastructure setup.

2. Types of Applications You Can Build with Spring Boot:

Spring Boot allows developers to create various types of applications easily:

- **Standalone Applications:** Create simple, self-contained Java applications that run on their own without requiring an external web server.
- **Web Applications:** Build web applications using embedded servers like Tomcat, Jetty, or Netty.
- **Distributed Applications:** Develop distributed systems or web services, including RESTful APIs, to interact with other applications.

Advantages of Using Spring Boot:

1. Starter POMs (Simplified Maven/Gradle Configuration):

Spring Boot provides a set of pre-configured "starter" dependencies that simplify your Maven or Gradle build configuration. These starters automatically configure common components, allowing you to focus on writing your business logic.

- **web-starter:** For creating web applications.
- **datajpa-starter:** For integrating Spring Data JPA.
- **security-starter:** For enabling Spring Security.
- **mail-starter:** For integrating email functionality.

2. Auto Configuration:

Spring Boot takes care of most of the configuration automatically based on the context of the application.

- **Database Connection Pooling:** Spring Boot automatically configures database connections, removing the need for manual setup.
- **Embedded Server Deployment:** Spring Boot can deploy the web application in an embedded server, eliminating the need for an external web server like Tomcat.
- **IOC Container Management:** The Spring Boot application will start the **Inversion of Control (IoC)** container and manage your beans automatically.
- **Component Scanning:** Spring Boot will scan for components, services, and repositories based on the configuration, reducing the need for manual bean registration.

3. Actuators:

Spring Boot includes **Actuators** that help you monitor and manage your application. These are essential for maintaining application health and performance.

- **Bean Count:** See how many beans are loaded in your application.
- **URL Mappings:** View all URL patterns mapped in the application.
- **Loaded Configuration Properties:** List all properties that are loaded in the application.
- **Application Health:** Check the health status of your application, including custom health checks.
- **Heap Dump:** Get a snapshot of the JVM memory.
- **Thread Dump:** Inspect the application threads for performance issues.

4. Embedded Servers:

Spring Boot comes with built-in support for various embedded web servers, eliminating the need for deploying your application to an external server.

- **Apache Tomcat (default):** Spring Boot comes pre-configured with Tomcat as the default embedded server.
- **Jetty:** Another lightweight and fast server option.
- **Netty:** A non-blocking I/O server designed for high-performance applications.

Creating a Spring Boot Application:

1. Initializr Website (start.spring.io):

You can create a Spring Boot application by visiting [Spring Initializr](https://start.spring.io), where you can generate a custom Spring Boot project with your desired dependencies and configuration.

2. IDE (Integrated Development Environment):

Many IDEs, such as **IntelliJ IDEA** and **Eclipse**, **STS**, provide direct integration with the Spring Initializr website, allowing you to create and configure Spring Boot applications seamlessly.

Note: IDEs internally connect to **start.spring.io** to generate the Spring Boot project (Spring Starter Project).

Core Annotations in Spring Framework

Spring provides a wide range of annotations that simplify configuration, improve flexibility, and enhance the ease of use for developers. Below is an overview of the core annotations used in Spring, categorized into **stereotype annotations**, **dependency injection annotations**, and others.

What is a Spring Bean?

A **Spring Bean** is any Java class (whether predefined, user-defined, or from a third-party library) whose object is created, managed, and maintained by the **Spring container**. Spring Beans are at the heart of Spring's IoC mechanism and undergo lifecycle management, including dependency injection.

What Does `@SpringBootApplication` Do?

The `@SpringBootApplication` annotation is a convenience annotation that combines three important annotations:

1. **`@SpringBootConfiguration`**: Equivalent to `@Configuration` in Spring, it marks the class as the configuration class for the Spring Boot application.
2. **`@EnableAutoConfiguration`**: Tells Spring Boot to automatically configure the application based on the dependencies present in the classpath.

3. **@ComponentScan**: Enables Spring to scan for components, configurations, and services in the package and sub-packages of the class.

Stereotype Annotations:

These annotations help in defining the roles or responsibilities of the beans within the Spring container. They are used to mark a class as a special type of Spring bean.

@Component

- **Purpose**: Marks a class as a Spring bean, indicating that it will be auto-detected through classpath scanning and instantiated in the Spring container.
- **Usage**: Use **@Component** when you want to create a generic Spring bean.

@Service

- **Purpose**: A specialization of **@Component**, used to annotate service-layer beans. It provides better readability and indicates that the class is holding business logic.
- **Usage**: Use **@Service** to mark service classes that contain business logic or service-related operations.

Uploaded by @coding_discussion2

@Repository

- **Purpose**: A specialization of **@Component**, used to annotate DAO (Data Access Object) classes. It marks the class as a data access object (typically responsible for interacting with the database).
- **Usage**: Use **@Repository** to annotate classes responsible for database operations.

@Controller

- **Purpose**: A specialization of **@Component**, used to mark classes that handle HTTP requests in Spring MVC applications. It is typically used for Spring Web MVC controller classes.
- **Usage**: Use **@Controller** to define a Spring MVC controller class.

@RestController

- **Purpose**: A combination of **@Controller** and **@ResponseBody**. It is used in RESTful web service development and automatically serializes response data to JSON or XML.
- **Usage**: Use **@RestController** for creating REST APIs in Spring MVC.

@Configuration

- **Purpose:** Indicates that a class is a source of bean definitions. This annotation is used for classes that define bean methods annotated with @Bean.
- **Usage:** Use @Configuration when you want to declare beans programmatically in Java (instead of XML).

Dependency Injection Annotations:

These annotations are used to inject dependencies into Spring beans, supporting Spring's Inversion of Control (IoC) container.

@Autowired

- **Purpose:** Automatically injects the dependency into a Spring bean, either by constructor, setter method, or field. Spring uses the @Autowired annotation to wire dependencies automatically.
- **Usage:** Use @Autowired to inject dependencies into Spring beans.

@Qualifier

- **Purpose:** This annotation is used in conjunction with @Autowired when multiple beans of the same type are available. It helps in specifying which bean should be injected when there are multiple candidates.
- **Usage:** Use @Qualifier to specify the exact bean to inject.

@Primary

- **Purpose:** Specifies that a bean is the default choice for autowiring when multiple candidates of the same type are present.
- **Usage:** Use @Primary to mark a preferred bean when multiple beans are available for autowiring.

Bean Definition Annotations:

@Bean

- **Purpose:** Indicates that a method produces a bean that should be managed by the Spring container.
- **Usage:** Use @Bean within a class annotated with @Configuration to define a bean.

@PostConstruct and @PreDestroy

- **Purpose:** These annotations are used for lifecycle management. `@PostConstruct` is executed after the bean's initialization, and `@PreDestroy` is executed before the bean is destroyed.
- **Usage:** Use `@PostConstruct` to define logic that needs to run after the bean's creation, and `@PreDestroy` to define cleanup logic.

@Profile

- **Purpose:** Specifies that a bean is only available in certain environments or profiles (e.g., "development", "production").
- **Usage:** Use `@Profile` to define beans that should be loaded only in specific profiles.

Uploaded by @coding_discussion2